ASHESI
UNIVERSITY

Implementation

Ashesi IRB Application Manager

Kelvin K. Ahiakpor

CS443: Mobile App Development

Dr. David Sampah

April 26, 2026

1 Implementation

This section explains how the Ashesi IRB Application Manager was implemented from end to end. The goal of the build was practical: make IRB review work faster for committee members, while giving students a simple way to submit applications and check status. The final solution includes a mobile reviewer app, a web submission/status portal, a Supabase backend, and notification pipelines for email, SMS, and push.

Live demo: https://youtu.be/RJ1JXf3P6So

Source code: https://github.com/kelvin-ahiakpor/irb_manager

2 Implementation stack

2.1 Programming languages

- Dart (Flutter application logic and UI)
- TypeScript (Supabase Edge Functions)
- SQL (Supabase schema, migrations, and policies)

2.2 Frameworks and platforms

- Flutter (cross-platform UI for Android, iOS, and Web)
- Supabase (Backend-as-a-Service)
- Vercel (deployment target for Flutter web build)
- Firebase Cloud Messaging (push delivery)

2.3 Database and backend services

- Supabase PostgreSQL database
- Supabase Auth (reviewer authentication support)
- Supabase Storage (private attachment bucket)
- Supabase Edge Functions (mail polling + notification workflows)

2.4 Core libraries and packages used

- supabase_flutter: client SDK for DB, auth, storage, functions
- go_router: route management and navigation
- hive_flutter: local cache and offline queue
- connectivity_plus: online/offline detection and sync triggers
- firebase_messaging + firebase_core: push token registration and permissions
- flutter_pdfview: in-app PDF rendering
- path_provider + dart:io: local persistent file caching
- file_picker (native) and dart:html input path (web): document upload
- url_launcher: open signed links in browser when needed

2.5 External APIs and integrations

- Microsoft Graph API (IRB mailbox polling and email parsing)
- Twilio API (SMS notifications)
- Resend API (email notifications)
- Firebase Cloud Messaging API (push notifications to reviewers)

3 High-level system implementation

The implementation follows a three-layer pattern:

- Presentation layer: Flutter screens for reviewer and student workflows
- Application layer: business logic for filtering, status updates, offline fallback, and synchronization
- Data and integration layer: Supabase database/storage plus Edge Functions for ingestion and notifications

On the reviewer side, committee members sign in, view incoming applications, open submitted files, update statuses, and write notes. On the student side, users can submit via web form and look up application progress by student ID.

When a submission enters the system, either from the web form or from the mailbox poller, it is written to the applications table, related attachment rows are saved, status history starts at PENDING, and notifications are triggered through configured channels.

4 Major implementation components

4.1 mobile application (Flutter)

- Main implemented screens/components:
- Splash screen with startup routing logic
- Microsoft login screen for reviewer authentication
- Reviewer dashboard with status filters (ALL, PENDING, IN REVIEW, CONDITIONALLY APPROVED, APPROVED, REJECTED)
- Application detail view with metadata, notes, and attachment list
- Document viewer for in-app PDF reading
- Update status sheet for decision actions and notes
- Status history timeline
- Settings screen

What was implemented:

- Real-time application list stream from Supabase
- Status filtering and quick reviewer workflow
- Notification trigger after status change
- Offline banners and fallback behaviors

4.2 Student web workflow (Flutter web)

Main implemented screens/components:

- Web submission form
- Student status lookup page

What was implemented:

- Form validation for required fields
- Multi-file attachment upload pipeline to Supabase Storage
- Insert of application and attachment metadata into database
- Fire-and-forget notification call after successful submission
- Public status lookup by student ID without account login

4.3 Backend and database implementation (Supabase)

Core tables implemented:

- reviewers
- applications
- attachments
- status_history
- notifications

Core backend logic implemented:

- Row-level access control policies for reviewer-safe operations
- Edge Function for mailbox polling and email ingestion
- Edge Function for email/SMS/push delivery
- Storage organization for per-application attachment paths

Database design decisions:

- Application records keep core student/research metadata
- Attachment metadata is decoupled from blob file storage
- Status changes are audit-trailed in status_history
- Notification events are logged for traceability

5 Major implementation components

A core part of this implementation was handling weak or unavailable connectivity for reviewers.

Implemented offline capabilities:

- Local cache of applications in Hive (key: applications)
- Local cache of reviewer identity for offline login bypass
- Offline queue for status updates (key: status_update_queue)
- Reconnect listener to flush queued updates to Supabase
- Attachment metadata caching for detail view
- Persistent local PDF file cache in application documents directory

Why this matters:

Reviewers can still open recently synced records, continue reading documents, and queue status decisions while offline. When the device reconnects, the app syncs queued updates automatically.

6 Notification pipeline implementation

The project supports three channels:

- Email via Resend
- SMS via Twilio

- Push via Firebase Cloud Messaging

Implementation behavior:

- New applications can trigger reviewer push notifications
- Status changes can trigger student email and SMS
- Notification attempts are recorded in notifications table
- Safe mock mode behavior exists when secrets are not configured

This setup made it possible to test notification behavior in development while still keeping production-ready paths for deployment.